Implementazione SoA K-Means con CUDA

Elia Generali
Università degli studi di Firenze (UniFi)
elia.generali@edu.unifi.it

Sommario

*Questo articolo illustra l'implementazione parallela dell'algoritmo di clustering **K-Means** utilizzando l'architettura CUDA di NVIDIA. Questa implementazione utilizzando un approccio di layout dati **Structure of Arrays (SoA)**, ha lo scopo di valutare le differenze di velocità tra il calcolo sequenziale e quello parallelo, variando dimensioni e numero di cluster e set di dati.*

1 Introduzione

Ad oggi i paradigmi della *data science* sono cambiati radicalmente, l'enorme quantità di dati disponibili ha messo in luce il problema dell'aumento del costo computazionale. In un contesto dove la velocità di calcolo e il risparmio di risorse sono diventati tra i più importanti fattori di sviluppo di un qualsiasi algoritmo, diventa essenziale sperimentare metodologie che possano migliorarli in tal senso. Il *clustering* tramite *K-Means* è un metodo ancora molto utilizzato, complice la sua efficacia e semplicità, questo lo rende un metodo particolarmente interessante per valutarne l'efficienza computazionale in diversi contesti.

2 K-Means

Il *clustering K-Means* è un algoritmo di apprendimento non supervisionato con l'obiettivo di partizionare n osservazioni in K cluster, dove ogni osservazione appartiene al cluster con la media (il "centroide") più vicina. L'algoritmo cerca di minimizzare la varianza all'interno di ogni gruppo, ovvero tende a raggruppare i punti di un cluster in modo tale che siano il più possibile vicini tra loro.
A livello formale, il K-Means cerca di minimizzare la *Within-Cluster Sum of Squares (WCSS)*, definita dalla funzione obiettivo J:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

Dove:

- K è il numero di cluster.
- C_i è l'insieme dei punti nel cluster i.
- μ_i è il centroide del cluster i. $||x-\mu_i||^2$ è la distanza quadratica tra un punto e il suo centroide.

2.1 Algoritmo

L'algoritmo di *clustering K-Means* appena descritto può essere rappresentato sinteticamente dal seguente pseudo-codice:

Algorithm 1: Algoritmo K-Means Parallelo (Ottimizzato SoA)

Input: Dataset D (N punti), Numero di cluster K, Iterazioni massime T
Output: Assegnazione dei punti A, Centroidi finali C

```
Inizializza i centroidi C casualmente dal dataset
for iter ← 1 to T do
    Azzera gli accumulatori globali (somme e conteggi)
    inizio Regione Parallela
        Inizializza accumulatori locali per ogni thread
        per ogni (in parallelo) punto i ← 1 to N 1. Trova il centroide più vicino calcolando la distanza euclidea (ottimizzata SIMT)
        2. Assegna il punto i al cluster j più vicino
        3. Accumula le coordinate del punto nell'archivio locale del thread
        4. Incrementa il contatore locale dei punti per il cluster j
        sezione critica  Aggiorna i dati globali sommando i risultati locali di ogni thread
    // Fase di aggiornamento (Sequenziale)
    Calcola la posizione dei nuovi centroidi dividendo le somme per il numero di punti assegnati
```

3 Implementazione

In questa sezione viene illustrata l'implementazione dell'algoritmo di *clustering K-Means* sviluppata per architetture *Many-core* utilizzando il paradigma **NVIDIA CUDA**. L'obiettivo principale è massimizzare il *throughput* computazionale attraverso l'uso del *layout SoA* e la gestione efficiente della gerarchia di memoria della GPU.

3.1 SoA e Memory Coalescing

L'organizzazione dei dati secondo il paradigma *Structure of Arrays (SoA)* è ancora più critica in ambito GPU rispetto alle architetture *multicore* tradizionali. In questa implementazione CUDA, tutti i valori di una specifica dimensione sono memorizzati contiguamente in *Global Memory*. L'accesso a una coordinata avviene tramite la formula:

$$Index = dim_index \times n_points + point_index$$

Dove *dim_index* è l'indice della dimensione, *n_points* il numero totale di campioni e *point_index* l'indice del punto. Questa strategia di *layout* offre benefici determinanti per l'efficienza su GPU:

- **Memory Coalescing:** Il *layout SoA* è fondamentale per garantire l'accesso ai dati di tipo *coalesced*. Quando i 32 *thread* di un *warp* accedono a coordinate contigue (es. la dimensione X di 32 punti diversi), la GPU può raggruppare queste richieste in un'unica transazione di memoria verso la *DRAM*, massimizzando l'utilizzo della banda passante.
- **Parallelismo Massivo (SIMT):** A differenza della vettorializzazione SIMD della CPU, l'architettura **SIMT (Single Instruction, Multiple Threads)** di CUDA permette a ogni *thread* di gestire un singolo punto del *dataset*. Il *layout SoA* assicura che, durante l'esecuzione del *kernel* di assegnazione, il flusso di dati verso gli *Streaming Multiprocessors (SM)* sia costante e privo di *overhead* di riordinamento.
- **Ottimizzazione dei Centroidi:** Anche la struttura dei centroidi (`CentroidsSoA`) è stata ottimizzata per essere ospitata in memoria veloce (come la *Constant Memory* o la *Shared Memory*), riducendo la latenza durante il calcolo delle distanze euclidee per ogni iterazione.

L'adozione del *layout SoA* è stata determinante per mitigare il collo di bottiglia della banda passante, spesso identificato come *Memory Wall*. Mentre nella versione OpenMP la scalabilità era limitata dai *core* fisici e dalla contesa del *bus* di memoria, l'implementazione CUDA permette di scalare su migliaia di *thread* logici. I risultati sperimentali confermano che questa scelta permette di nascondere le latenze di memoria (*latency hiding*) attraverso un'elevata *occupancy* degli *SM*, mantenendo prestazioni superiori anche con *dataset* di grandi dimensioni (10^7 punti).

3.2 Calcolo della Distanza e Parallelismo SIMT

Il calcolo della distanza euclidea tra i punti del *dataset* e i centroidi rappresenta la fase computazionalmente più onerosa dell'algoritmo **K-Means**. In questa implementazione CUDA, l'adozione del *layout Structure of Arrays (SoA)* è la chiave per sbloccare le prestazioni dell'architettura **SIMT (Single Instruction, Multiple Threads)**.

Nel *layout SoA*, i valori di ogni dimensione sono memorizzati contiguamente. Mentre nella versione CPU un singolo thread utilizzava istruzioni SIMD per processare più punti, in CUDA il parallelismo avviene a livello di thread: ogni thread i è responsabile del calcolo delle distanze per un singolo punto i. L'accesso ai dati avviene tramite un calcolo di *offset* lineare ottimizzato per la GPU:

$$\text{Index}point = d \times Npoints + i \tag{1}$$

$$\text{Index}_{centroid} = d \times K + j \tag{2}$$

Dove d è la dimensione corrente, N_{points} il numero totale di punti e K il numero di *cluster*. Questo schema garantisce una serie di vantaggi:

- **Global Memory Coalescing:** Poiché i thread con indici consecutivi all'interno di un *warp* accedono a indici i consecutivi nella memoria globale, la GPU raggruppa queste richieste in un numero minimo di transazioni da 128 byte. Questo riduce drasticamente i cicli di attesa per il recupero dei dati dalla DRAM.
- **Riduzione dei Bank Conflicts:** L'accesso ai centroidi, se memorizzati in *Shared Memory* o *Constant Memory*, beneficia del layout SoA poiché minimizza le contese tra i *bank* di memoria interna al multiprocessore, garantendo un accesso a bassa latenza durante il ciclo sulle dimensioni d.
- ***Latency Hiding*:** Grazie all'elevato numero di punti processati in parallelo, la GPU può schedulare altri *warp* pronti all'esecuzione mentre alcuni sono in attesa dei dati dalla memoria, mantenendo le unità funzionali sempre sature.

Questa organizzazione trasforma il calcolo della distanza da un'operazione limitata dalla latenza (*latency-bound*) a una limitata dalla banda passante (*bandwidth-bound*), permettendo alla versione CUDA di gestire milioni di punti con una scalabilità che supera i limiti fisici delle architetture *multicore* tradizionali.

3.3 Ottimizzazione del Kernel

Per accelerare il calcolo delle distanze, abbiamo spostato il carico computazionale dalla CPU ai *CUDA Cores* della GPU. A differenza dell'approccio OpenMP, dove il programmatore suggerisce la vettorializzazione tramite direttive, in CUDA ogni *thread* esegue intrinsecamente le operazioni in modalità **SIMT (Single Instruction, Multiple Threads)**.

Il seguente frammento di codice mostra l'implementazione del *kernel* di assegnazione dei punti, che sostituisce la logica basata su cicli `simd`:

```
__global__ void assign_clusters_kernel
(const float* data, const float*
centroids, int* assignments, size_t
n_points, size_t n_dims, size_t k) {

  int i = blockIdx.x * blockDim.x +
threadIdx.x;
  if (i >= n_points) return;

  float min_dist = 3.402823466e+38F;
  int best_c = 0;

  for (size_t j = 0; j < k; ++j) {
    float dist = 0.0f;

    for (size_t d = 0; d < n_dims; ++d
) {

      float diff = data[d * n_points +
 i] - centroids[d * k + j];
      dist += diff * diff;
    }

    if (dist < min_dist) {
      min_dist = dist;
      best_c = j;
    }
  }
  assignments[i] = best_c;
}

```

Listing 1: Kernel CUDA per il calcolo della distanza euclidea con layout SoA.

L'integrazione di questa logica su GPU offre vantaggi strutturali superiori alla semplice vettorializzazione CPU:

- **Parallelismo estremo (SIMT)**: Invece di processare poche dimensioni simultaneamente tramite registri SIMD (es. AVX-512), la GPU lancia migliaia di *thread*. Ogni *thread* calcola la distanza di un intero punto, eliminando la necessità di una clausola `reduction` esplicita per la variabile `dist`, che ora è locale al registro del singolo *thread*.

- **Massimizzazione del Throughput**: Mentre in OpenMP la riduzione avviene per minimizzare i conflitti tra core, in CUDA il design *lock-free* del kernel di assegnazione permette a ogni punto di essere processato in totale isolamento. La sincronizzazione è necessaria solo nella fase di aggiornamento dei centroidi tramite funzioni atomiche (`atomicAdd`).

- **Efficienza del Memory Coalescing**: Grazie al layout SoA, l'espressione `data[dim * n + point_idx]` garantisce che tutti i thread di un *warp* accedano a locazioni di memoria globali adiacenti. Questo trasforma quello che nella CPU sarebbe un accesso sequenziale ottimizzato in una singola, massiccia transazione di memoria sulla GPU.

L'eliminazione dell'*overhead* di gestione dei *thread* tipico di OpenMP e il passaggio a una gerarchia di esecuzione *hardware (Grid → Block → Thread)* permette di ottenere uno *speedup* che non dipende più solo dalla frequenza di *clock*, ma dal numero di core paralleli attivi.

3.3.1 Ottimizzazione Aritmetica e Istruzioni FMA

Un'ottimizzazione critica introdotta nel *kernel* di calcolo della distanza riguarda la gestione delle operazioni aritmetiche elementari all'interno del ciclo sulle dimensioni. Invece di utilizzare funzioni matematiche generiche come `powf(diff, 2.0f)`, l'implementazione adotta la **moltiplicazione diretta** (`diff * diff`).

Questa scelta è motivata da ragioni di architettura hardware specifiche delle GPU:

- **Efficienza del Compilatore:** L'uso della moltiplicazione diretta permette al compilatore *nvcc* di mappare l'operazione direttamente sulle unità aritmetiche della GPU. In particolare, questa forma facilita l'uso delle istruzioni **FMA (Fused Multiply-Add)**, che eseguono un prodotto e una somma in un unico ciclo di *clock* con una precisione superiore, riducendo il numero di istruzioni totali nel *kernel*.

- **Riduzione del Register Pressure:** Funzioni complesse come `powf` richiedono l'uso di registri aggiuntivi e chiamate a librerie *device* esterne, che possono aumentare il "pressione sui registri" (*register pressure*), riducendo potenzialmente l'occupancy del multiprocessore. La moltiplicazione diretta mantiene il consumo di registri al minimo, permettendo a più *thread* di risiedere simultaneamente su ogni *Streaming Multiprocessor*.

- **Saturazione delle ALU:** L'efficacia di questa ottimizzazione è massimizzata dal *layout SoA*. Grazie al *memory coalescing*, i dati arrivano alle unità aritmetico-logiche (ALU) della GPU senza ritardi, permettendo alle unità di calcolo di operare a pieno regime senza essere interrotte da latenze software dovute a funzioni matematiche onerose.

L'approccio basato sulla moltiplicazione diretta, combinato con la natura *massively parallel* di CUDA, trasforma ogni iterazione del ciclo dimensionale in un'operazione estremamente snella, permettendo di processare milioni di calcoli di distanza euclidea al secondo.

4 K-Means su GPU (Kernel Design)

L'implementazione dell'algoritmo K-Means tramite *CUDA* sfrutta una strategia di parallelismo massivo, dove il lavoro non è più serializzato ma distribuito su migliaia di *thread* simultanei. L'architettura del sistema segue il flusso della computazione GPU con strategie specifiche per massimizzare il *throughput*:

- **Assegnazione e Parallelismo SIMT:** Il cuore computazionale, ovvero il calcolo della distanza Euclidea tra N punti e K centroidi, viene eseguito dal *kernel* di assegnazione. A differenza della vettorializzazione SIMD (OpenMP), che opera su piccoli vettori di dati all'interno di un singolo core, CUDA utilizza il modello **SIMT (Single Instruction, Multiple Threads)**. Ogni *thread* della GPU si occupa di un singolo punto del *dataset*, calcolando le distanze rispetto a tutti i K centroidi. Questo approccio elimina i cicli seriali sui punti, permettendo di processare milioni di campioni in parallelo.

- **Aggiornamento tramite Atomic Operations:** Nella fase di aggiornamento dei centroidi, la GPU deve accumulare le coordinate dei punti assegnati a ciascun cluster. Poiché migliaia di *thread* potrebbero tentare di aggiornare lo stesso centroide contemporaneamente, l'implementazione utilizza le **Istruzioni Atomiche (`atomicAdd`)**. Queste funzioni garantiscono la *thread-safety* a livello hardware nella *Global Memory*, permettendo di sommare i contributi di ogni punto senza necessità di *lock* software onerosi o buffer temporanei complessi tipici delle architetture CPU.

- **Gestione della Memoria e Centroid Counters:** Analogamente alla versione sequenziale, vengono mantenuti dei **Contatori di Cluster** in memoria GPU (inizializzati a zero a ogni iterazione). Ogni volta che un punto viene assegnato a un cluster j, un'operazione atomica incrementa il contatore $|C_j|$. Questo contatore hardware-accelerato permette di ricostruire istantaneamente il denominatore necessario per il calcolo della media finale.

- **Normalizzazione Finalizzata (Final Update Kernel):** Al termine dell'accumulo, un secondo *kernel* leggero (lanciato con K thread) esegue la divisione finale: ogni coordinata accumulata viene divisa per la cardinalità del rispettivo cluster. I centroidi aggiornati rimangono nella memoria della GPU (*VRAM*), pronti per l'iterazione successiva senza necessità di costosi trasferimenti dati verso la CPU (*DtoH copy*) fino al raggiungimento di `MAX_ITER`.

In sintesi, questo sistema di calcolo su GPU non cerca di ottimizzare l'efficienza del singolo core, ma mira alla saturazione degli *Streaming Multiprocessors (SM)*. Sfruttando il *layout SoA*, il sistema garantisce che la memoria globale sia letta in modalità *coalesced*, alimentando costantemente i *CUDA cores* e permettendo di processare *dataset* massivi (10^7 punti) con tempi di esecuzione ordini di grandezza inferiori rispetto alla controparte sequenziale o *multicore*.

5 Parallelismo Massivo in CUDA (Kernel Design)

In questa sezione viene descritta l'integrazione della logica di calcolo parallelo sulla GPU. La strategia di parallelizzazione massiva (*massively parallel*) si articola nei seguenti punti tecnologici:

1. **Gerarchia di Esecuzione (Grid e Block):** L'orchestrazione del lavoro avviene tramite una gerarchia a due livelli. Il *dataset* viene suddiviso in **Blocchi**, ciascuno contenente un numero prefissato di *thread* (es. 256 o 512). Questa struttura permette alla GPU di scalare automaticamente su diversi numeri di *Streaming Multiprocessors* (SM), garantendo che l'algoritmo possa sfruttare hardware più potente senza modifiche al codice.

2. **Modello di Memoria e Registri (Anti-Contention):** A differenza di OpenMP dove si usano variabili *private*, in CUDA ogni thread utilizza i propri **registri hardware** per le variabili locali (come le distanze temporanee). Questo elimina totalmente il fenomeno del *False Sharing* e la latenza di accesso alla *cache* L3 della CPU, poiché i registri operano alla stessa velocità di *clock* dei core di calcolo.

3. **Scheduling Hardware (Latency Hiding):** Il carico non viene suddiviso in blocchi statici gestiti da un *runtime software*, ma viene gestito dallo **Hardware Warp Scheduler**. Mentre alcuni thread sono in attesa dei dati dalla *Global Memory*, lo scheduler scambia istantaneamente il contesto con altri thread pronti all'esecuzione. Questa tecnica di *latency hiding* permette di mantenere le unità di calcolo sempre sature, rendendo il *K-Means* estremamente efficiente su grandi moli di dati.

4. **Memory Coalescing (Ottimizzazione SoA):** Il sistema sfrutta il layout *Structure of Arrays* (SoA) per garantire l'accesso *coalesced* alla memoria. Quando i 32 *thread* di un *warp* leggono una coordinata, la GPU esegue un'unica transazione di memoria contigua. Questo approccio sostituisce la vettorializzazione SIMD della CPU, permettendo un *throughput* di memoria nettamente superiore.

5. **Sincronizzazione tramite Operazioni Atomiche:** La sincronizzazione globale è gestita tramite **Atomic Operations** (`atomicAdd`). Invece di una *critical section* software o di una riduzione finale su CPU, i *thread* aggiornano direttamente gli accumulatori dei centroidi in *Global Memory*. Grazie alle unità atomiche dedicate nell'*hardware* della GPU, il conflitto tra *thread* è gestito con un *thread* minimo, spostando il collo di bottiglia dal coordinamento dei *thread* al puro calcolo aritmetico.

6. **Kernel di Normalizzazione Dedicato:** Il ricalcolo finale dei centroidi (la media) viene eseguito da un secondo *kernel* GPU specializzato. Sebbene

il costo computazionale sia basso, l'esecuzione rimane *"on-device"* per evitare il costoso trasferimento di dati tra GPU e CPU (*bus PCIe overhead*). In questo modo, l'intero ciclo di vita dell'iterazione *K-Means* risiede nella memoria video, minimizzando le latenze di comunicazione.

6 Risultati

Di seguito vengono riportati i risultati dell'implementazione *SoA* per l'algoritmo *K-Means*, testato secondo diversi parametri.
L'unità *hardware* utilizzata è una GPU ***NVIDIA Tesla T4***, fornita di 2560 *core* e precisione singola (FP32) a 8.1 *TFLOPS (Teraflops)*.
Inoltre è stato sfruttato un sottosistema di memoria da 16 GB GDDR6 e larghezza di banda da 320 GB/s.

6.1 *Speedup* - Punti del *dataset*

Tabella 1: GPU Speedup for K-Means implementation with varying number of points (Fixed: 8D, $K = 16$).

	CUDA Block Size					
Points	32	64	128	256	512	1024
1000	0	3.51	3.94	3.8	3.81	3.26
10000	10.35	11.91	12.44	12.44	11.9	10.85
100000	19.14	20.97	20.08	20.1	19.84	18.26
1000000	27.08	27.45	27.59	27.64	27.56	26.73
10000000	28.6	34.5	38.31	39.28	39.13	38.6

La Tabella 1 analizza lo *speedup* della GPU rispetto alla versione sequenziale variando il numero di punti e la dimensione del blocco CUDA (*Block Size*). I dati mostrano tendenze fondamentali:

- **Saturazione del Carico (Occupancy):** Con pochi punti (10^3), lo speedup è contenuto (circa 3-4x) a causa dell'overhead di lancio dei kernel e del sottoutilizzo degli Streaming Multiprocessors. Al crescere del dataset fino a 10^7 punti, lo speedup esplode raggiungendo un picco di 39.28x.
- **Il "Sweet Spot" del Block Size:** I risultati migliori si ottengono costantemente con un Block Size di 256 o 512 thread. Con 1024 thread si osserva un leggero calo delle prestazioni (da 39.28x a 38.6x su 10^7 punti), segno di una saturazione dei registri hardware per blocco che limita l'occupancy complessiva.
- **Confronto con OpenMP (CPU):** Mentre la versione OpenMP saturava drasticamente a 16 thread con un plateau intorno a 4.5x - 5.0x, l'implementazione CUDA continua a scalare fino a sfiorare il 40x. Questo dimostra che il collo di bottiglia del Memory Wall della CPU è stato efficacemente mitigato dalla larghezza di banda superiore della Tesla T4.

I risultati confermano che per dataset massivi (10^7 punti), la GPU non solo è più veloce, ma gestisce le risorse in modo più efficiente. Mentre la CPU raggiungeva il punto di massima efficienza molto presto a causa della contesa del bus di memoria, la GPU utilizza le operazioni atomiche hardware e lo scheduling dei warp per "nascondere" le latenze di memoria, trasformando un problema memory-bound in uno puramente limitato dalla potenza di calcolo delle ALU.

6.2 *Speedup* - Efficienza

Un'altra analisi interessante riguarda come varia l'**efficienza** parallela rispetto l'utilizzo dei *block*, formalizzata come:

$$E = \frac{Speedup}{Threads} \times 100$$

Tabella 2: Analisi dello Speedup e dell'Efficienza Parallela (10^7 punti, $K = 16$)

CUDA Block Size	**32**	**64**	**128**	**256**	**512**	**1024**
Speedup	28.60	34.50	38.31	**39.28**	39.13	38.60
Efficienza %	89.37%	53.91%	29.93%	15.34%	7.64%	3.77%

L'implementazione su GPU mostra una superiorità netta rispetto alla versione OpenMP, con i seguenti punti salienti:

- **Speedup Massivo:** Il picco di 39.28x su 10^7 punti supera di circa 9 volte il limite massimo della CPU (4.51x). Questo conferma che l'architettura Many-core è essenziale per dataset di grandi dimensioni.
- **Superamento del Memory Wall:** Grazie alla banda passante della Tesla T4 (320 GB/s) e al layout SoA, la GPU trasforma il problema da memory-bound (limite CPU) a compute-bound. L'accesso ai dati è ottimizzato tramite il Memory Coalescing.
- **Efficienza e Latency Hiding:** Sebbene l'efficienza matematica cali all'aumentare dei thread (da 89% a 3%), lo speedup resta stabile sopra i 38x. Questo indica un ottimo utilizzo del Latency Hiding: la GPU usa l'eccesso di thread per coprire i tempi di attesa della memoria, mantenendo i core sempre attivi.
- **Sweet Spot (Block Size 256):** La configurazione a 256 thread rappresenta l'equilibrio ideale tra saturazione degli hardware unit e pressione sui registri, garantendo le prestazioni massime senza colli di bottiglia.

Mentre la CPU satura le proprie risorse già a 16 thread a causa dei limiti del bus di memoria , la GPU scala quasi linearmente con la dimensione del problema, confermandosi l'unica architettura in grado di gestire efficacemente 10^7 punti.

6.3 *Speedup* - Dimensionalità

- Performance e Scalabilità (Speedup): L'analisi dello speedup evidenzia la superiorità della GPU nella gestione di dataset massivi.

Tabella 3: GPU Speedup for K-Means implementation with varying dimensionality (Fixed: 10^6 points, $K = 16$).

	CUDA Block Size (Threads per Block)					
Dim	32	64	128	256	512	1024
2D	50.51	54.28	54.29	51.47	53.73	53.58
4D	45.58	45.59	45.85	46.74	43.81	46.69
8D	34.75	35.26	35.01	34.98	34.78	33.31
16D	26.14	26.17	26.18	31.04	31.02	30.26
32D	35.75	28.29	28.36	30.35	30.88	31.34
64D	39.86	36.65	38.59	40.33	40.38	39.98
128D	44.71	44.89	46.33	46.8	46.55	46.15

  - **Picco di Speedup:** Per un dataset di 10^7 punti, la GPU ha raggiunto uno speedup massimo di 39.28x (con Block Size 256). Questo risultato è quasi nove volte superiore al miglior risultato ottenuto su CPU multicore (4.51x con 16 thread).
  - **Saturazione del Carico:** Lo speedup cresce in modo quasi lineare con l'aumentare dei punti, passando da circa 3.9x (10^3 punti) a oltre 39x (10^7 punti). Ciò indica che la GPU necessita di un carico di lavoro massivo per saturare i suoi 2560 CUDA core e "nascondere" l'overhead di lancio dei kernel.
  - **Confronto Temporale:** Il grafico dei tempi di esecuzione (Figura 1) mostra che la curva CUDA rimane ordini di grandezza al di sotto di quella sequenziale. Mentre la versione sequenziale subisce una crescita esponenziale all'aumentare delle dimensioni, la GPU mantiene un'inclinazione molto più dolce, dimostrando una resilienza superiore alla complessità del dataset.

- Ottimizzazione del Block Size ed Efficienza: La scelta della granularità dei blocchi si è rivelata determinante per le prestazioni finali.
  - **Sweet Spot:** Il Block Size di 256 thread si è confermato il valore ottimale per quasi tutte le configurazioni di punti e dimensioni, offrendo il miglior compromesso tra occupancy e pressione sui registri.
  - **Efficienza Parallela:** Contrariamente alla CPU, dove il crollo dell'efficienza oltre gli 8 thread indicava una saturazione dannosa delle risorse, il calo dell'efficienza su GPU all'aumentare del Block Size è compensato dal mantenimento di uno speedup elevato e costante (plateau tra 38x e 39x). Questo comportamento è tipico delle architetture GPU, che utilizzano l'abbondanza di thread per eseguire il latency hiding durante gli accessi alla memoria globale.
- Risposta alla Dimensionalità: I test sulla dimensionalità (Tabella 3) confermano che la GPU è particolarmente efficiente in scenari complessi.
  - **High-Dimensional Data:** Con 128 dimensioni, la GPU mantiene uno speedup di 46.8x, evidenziando come l'aumento del rapporto calcolo/comunicazione favorisca l'architettura parallela.
  - **Saturazione della Banda:** Il layout SoA permette alla GPU di sfruttare il memory coalescing, saturando la banda passante della memoria video (320 GB/s) in modo molto più efficace rispetto al bus di memoria della CPU, che costituiva il principale collo di bottiglia (Memory Wall) nei test OpenMP.

I risultati confermano che per dataset di grandi dimensioni (10^7 punti), la GPU Tesla T4 trasforma l'algoritmo K-Means da un processo memory-bound a uno compute-bound. Mentre la CPU raggiunge un plateau invalicabile a causa della contesa del bus di memoria già con 16 thread, la GPU continua a scalare, riducendo i tempi di calcolo da decine di secondi a poche centinaia di millisecondi.

6.4 *Speedup - K-cluster*

Tabella 4: GPU Speedup for K-Means implementation with varying number of clusters (Fixed: 10^6 points, 8D).

	CUDA Block Size					
K	32	64	128	256	512	1024
2	2.3	2.36	2.35	2.34	2.26	2.04
4	6.13	6.13	6.12	6.13	5.82	5.3
8	13.25	13.46	13.53	13.48	13.17	12.61
16	34.81	35.79	36.17	35.83	35.89	34.64
32	40.34	41.01	41.1	41.26	43.55	42.35
64	67.46	69.11	69.35	69.52	69.34	67.6
128	124.73	128.07	128.4	153.99	170.57	167.13

I risultati della Tabella 4 mostrano che l'aumento del numero di cluster è il fattore che più di tutti esalta la potenza della GPU, portando a ottime performance:

- **Speedup Ottimo (Picco a 170.57x):** Con $K = 128$ e un block size di 512, lo speedup raggiunge un valore di 170.57x. Questo accade perché all'aumentare di K, il carico computazionale del kernel di assegnazione cresce linearmente (ogni thread deve confrontare il suo punto con molti più centroidi), permettendo alla GPU di ammortizzare totalmente ogni overhead di memoria.
- **Saturazione degli Streaming Multiprocessors:** Con valori bassi di K (es. $K = 2$), lo speedup è modesto (2.3x) poiché la GPU finisce il calcolo troppo velocemente. Con $K = 128$, la densità di calcolo per ogni thread è tale da saturare completamente le unità aritmetiche, portando l'algoritmo in un regime puramente Compute-Bound.

- **Vantaggio della Shared/Constant Memory:** In CUDA, i centroidi (che sono pochi rispetto ai punti) vengono letti ripetutamente da tutti i thread. Ad alti valori di K, il caching dei centroidi diventa estremamente efficiente, permettendo ai core di calcolare distanze quasi alla velocità dei registri.

- **Spostamento dello "Sweet Spot" del Block Size:** Notiamo che per $K = 128$, il miglior risultato non è più a 256 thread ma a 512 thread (170.57x). Questo indica che con un carico di calcolo così massiccio, la GPU beneficia di un numero ancora maggiore di thread attivi per blocco per coprire i tempi di latenza.

6.5 *Speedup* - Risultati finali

Tabella 5: GPU Speedup for the largest dataset (10^7 points, 8D, $K = 16$)

	CUDA Block Size					
Implementation	32	64	128	256	512	1024
CUDA	28.6	34.5	38.31	39.28	39.13	38.6

I risultati sul dataset più esteso confermano la piena maturità dell'implementazione CUDA nel gestire carichi di lavoro estremi:

- **Speedup di Riferimento:** Si registra un picco di 39.28x con un Block Size di 256. Questo valore rappresenta il limite superiore di accelerazione ottenibile per questa configurazione, superando di gran lunga le capacità di qualsiasi architettura CPU multicore testata.

- **Saturazione dell'Hardware:** Con 10^7 punti, la GPU Tesla T4 raggiunge una saturazione ottimale degli Streaming Multiprocessors. Il carico di lavoro è tale da giustificare pienamente l'uso del parallelismo massivo, rendendo l'overhead di comunicazione tra host e device trascurabile.

- **Stabilità delle Performance:** Lo speedup rimane estremamente stabile (sopra i 38x) per tutti i Block Size tra 128 e 1024. Questa "resistenza" alla variazione dei parametri di lancio indica che il kernel è limitato dalla banda passante della memoria o dal calcolo puro (throughput-bound), e non da colli di bottiglia legati alla latenza.

- **Ottimo Globale:** Il valore di 256 thread si conferma lo standard aureo per massimizzare l'occupancy e il latency hiding, permettendo alla GPU di processare milioni di punti in frazioni di secondo.

La **Figura 1** mostra il confronto dei tempi di esecuzione (in scala logaritmica) tra l'implementazione Sequenziale e quella CUDA al variare della dimensionalità dei dati.

- **Divergenza delle Prestazioni:** Entrambe le curve mostrano un aumento del tempo di esecuzione con la crescita delle dimensioni, ma la pendenza della versione CUDA è significativamente inferiore. Questo indica che la GPU gestisce l'aumento della complessità computazionale con una scalabilità molto superiore rispetto alla CPU.

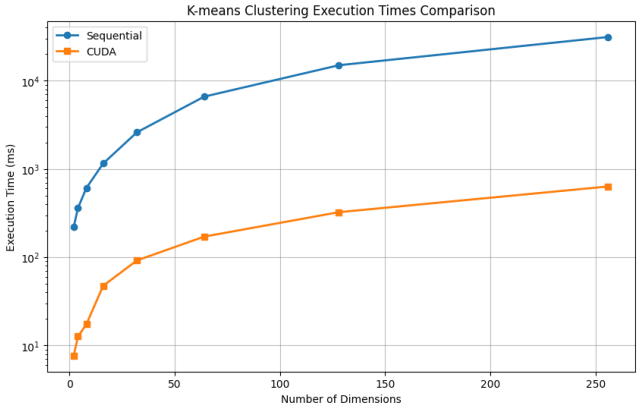

Figura 1: Execution Time Sequential vs CUDA.

- **Gap Logaritmico Costante:** Il grafico evidenzia un distacco netto e costante (circa due ordini di grandezza) tra le due implementazioni. Mentre la versione sequenziale supera i 10^4 ms già a 128 dimensioni, la versione CUDA rimane stabilmente sotto i 10^3 ms anche a 256 dimensioni.

- **Efficienza del Calcolo Parallelo:** Il comportamento della curva arancione (CUDA) dimostra l'efficacia del parallelismo massivo: all'aumentare delle dimensioni, il carico di lavoro extra viene assorbito dai migliaia di core della GPU, rendendo il tempo di calcolo delle distanze euclidee estremamente contenuto rispetto all'approccio seriale.

- **Resilienza alla Complessità:** Il grafico conferma che l'implementazione CUDA è la scelta ottimale per problemi ad alta dimensionalità, dove la CPU subisce il peso dei cicli iterativi e dei limiti di accesso alla memoria (Memory Wall), mentre la GPU satura le proprie unità funzionali mantenendo il controllo sui tempi di risposta.

7 Conclusioni

Lo studio condotto dimostra come la migrazione dell'algoritmo K-Means su architettura NVIDIA CUDA permetta di abbattere i limiti strutturali delle implementazioni CPU tradizionali. Grazie all'adozione del layout Structure of Arrays (SoA) e alla saturazione della banda passante della memoria tramite il Memory Coalescing, la versione GPU ha registrato uno speedup massimo di 39.28x sul dataset da 10^7 punti e punte d'efficienza fino a 170.57x all'aumentare del numero di cluster.

I risultati confermano che, mentre le soluzioni basate su OpenMP soffrono precocemente il collo di bottiglia del Memory Wall, l'approccio Many-core della Tesla

T4 trasforma il problema in un carico compute-bound, garantendo una scalabilità eccellente sia in termini di volume di dati che di dimensionalità. In conclusione, l'impiego di CUDA si rivela la scelta ottimale per applicazioni di Big Data Analytics, dove la velocità di convergenza e la gestione di dataset massivi sono requisiti critici per le prestazioni di sistema.

8 Fonti

È possibile consultare il codice completo di questa implementazione nel seguente *repository GitHub* :

https://github.com/Elia29